ORIENTATION IA

Plateforme d'Orientation Scolaire et Professionnelle

DOCUMENTATION TECHNIQUE DE L'API REST

À l'attention de l'équipe Frontend

Version 1.0 — Juillet 2026

Table des matières

1. Présentation du projet
2. Technologies utilisées
3. Authentification
4. Structure de la base de données
5. Liste complète des endpoints API
    5.1 Authentification
    5.2 Profil utilisateur
    5.3 Parcours académique
    5.4 Tests de personnalité
    5.5 Tests QI
    5.6 Analyse du métier rêvé
    5.7 Orientation et carrière
    5.8 Abonnements
    5.9 Rapports PDF
    5.10 Assistant IA
6. Schémas des requêtes et réponses
7. Codes d'erreur
8. Abonnements (Premium)

Annexe : Bonnes pratiques

1. Présentation du projet

Orientation IA est une plateforme web innovante qui aide les utilisateurs à faire des choix éclairés concernant leur orientation scolaire et professionnelle. Grâce à une combinaison de tests psychométriques (personnalité et QI), d'analyse de CV, d'études de marché et d'algorithmes de matching, la plateforme fournit des recommandations personnalisées et des feuilles de route carrière adaptées à chaque profil.

Cette documentation est destinée à l'équipe Frontend afin de faciliter l'intégration avec l'API REST. Elle décrit l'ensemble des endpoints disponibles, les schémas de données attendus et retournés, ainsi que les règles métier à respecter.

Architecture générale

- **Backend :** API RESTful développée avec Laravel 12.x (PHP 8.2)
- **Authentification :** Jetons d'accès (tokens) via Laravel Sanctum
- **Base de données :** SQLite (migrable vers MySQL/PostgreSQL)
- **Génération PDF :** Bibliothèque DomPDF pour les rapports
- **Frontend :** Application indépendante (Vue.js / React / Angular)

2. Technologies utilisées

Technologie	Usage	Version
PHP	Langage serveur	8.2+
Laravel	Framework PHP	12.x
Laravel Sanctum	Authentification API (tokens)	-
SQLite	Base de données	3.x
DomPDF	Génération de PDF	-
Vite	Build frontend	6.x
Tailwind CSS	Styles templates PDF	4.x

Note : L'API retourne toutes ses réponses en JSON. Les messages sont en français.

3. Authentification

3.1 Principe

L'API utilise **Laravel Sanctum** pour l'authentification par jeton (token). Chaque utilisateur reçoit un token Bearer lors de l'inscription ou de la connexion. Ce token doit être inclus dans le header `Authorization` de chaque requête protégée.

Header requis : Authorization: Bearer <token>

3.2 Inscription

Endpoint : `POST /api/register`

Corps de la requête :

```
{"name": "Jean Dupont", "email": "jean@example.com", "password": "motdepasse123",
"password_confirmation": "motdepasse123"}
```

Réponse (201 Created) :

```
{"user": {"id": 1, "name": "Jean Dupont", "email": "jean@example.com"}, "token": "1|abc123..."}
```

3.3 Connexion

Endpoint : `POST /api/login`

Corps :

```
{"email": "jean@example.com", "password": "motdepasse123"}
```

Réponse (200 OK) :

```
{"user": {"id": 1, "name": "Jean Dupont", "email": "jean@example.com"}, "token": "1|abc123..."}
```

3.4 Déconnexion

Endpoint : `POST /api/logout` Auth

Réponse : `{"message": "Déconnexion réussie."}`

3.5 Utilisateur courant

Endpoint : `GET /api/me` Auth

Retourne l'utilisateur avec son profil et ses diplômes.

Règle importante :

Le token doit être stocké côté client (httpOnly cookie de préférence, ou localStorage).
Toutes les routes sauf `/register` et `/login` nécessitent un token valide.

4. Structure de la base de données

La base de données contient 21 tables. Voici le détail des principales.

4.1 Table : users

Stocke les comptes utilisateurs avec leur statut d'abonnement.

Relations :

- Un utilisateur a UN profil (*user_profiles*)
- Un utilisateur a PLUSIEURS diplômes (*academic_records*)
- Un utilisateur a PLUSIEURS tests de personnalité (*personality_tests*)
- Un utilisateur a PLUSIEURS tests QI (*qi_tests*)
- Un utilisateur a PLUSIEURS analyses de métier rêvé (*dream_job_analyses*)
- Un utilisateur a PLUSIEURS abonnements (*subscriptions*)
- Un utilisateur a PLUSIEURS rapports PDF (*reports*)
- Un utilisateur a PLUSIEURS feuilles de route (*career_roadmaps*)

Colonne	Type	Description
id	bigint PK	Identifiant unique
name	string(255)	Nom complet
email	string(255) UNIQUE	Adresse email
password	string(255)	Mot de passe (hashé)
subscription_status	string	'free' ou 'premium'
subscription_expires_at	datetime NULL	Date d'expiration premium
phone	string NULL	Téléphone
avatar	string NULL	URL de l'avatar

4.2 Table : user_profiles

Stocke les informations personnelles et les préférences de carrière.

Relation : belongsTo(users)

Colonne	Type	Description
user_id (FK)	bigint	Référence vers users (cascade)
age	integer NULL	Âge
gender	string NULL	Genre
city, region, country	string NULL	Localisation
budget	decimal(12,2)	Budget pour formation
budget_currency	string(3)	Devise (EUR par défaut)
target_job_title	string NULL	Métier cible
target_job_description	text NULL	Description du métier cible

4.3 Table : academic_records

Stocke l'historique des diplômes et formations.

Relation : belongsTo(users)

Colonne	Type	Description
diploma	string(255)	Intitulé du diplôme
field_of_study	string(255)	Domaine d'étude
institution	string NULL	Établissement
grade	string NULL	Note ou mention
start_date / end_date	date NULL	Période
disliked_subjects	json NULL	Matières non appréciées
sort_order	integer	Ordre d'affichage

4.4 Table : personality_tests

Stocke les sessions de test de personnalité.

Relations : belongsTo(users) — hasMany(personality_trait_results)

Colonne	Type	Description
status	enum('in_progress','completed')	Statut du test
overall_score	integer NULL	Score global (moyenne)
summary	json NULL	Résumé par trait
started_at / completed_at	datetime	Horodatage

4.5 Table : personality_trait_results

Stocke les scores individuels pour chaque trait de personnalité.

Relation : belongsTo(personality_tests)

Colonne	Type	Description
personality_test_id (FK)	bigint	Référence vers le test
trait_name	string(100)	Nom du trait
score	integer	Score de 0 à 100
details	json NULL	Détails supplémentaires

4.6 Table : qi_tests

Stocke les sessions de test de QI avec score calculé.

Relations : belongsTo(users) — hasMany(qi_subsections, qi_answers)

Colonne	Type	Description
total_score	integer NULL	Score total
iq_score	integer NULL	QI calculé (70-130)
percentile	decimal(5,2) NULL	Rang percentile
standard_deviation	decimal(5,2) NULL	Écart-type

4.7 Table : qi_questions

Banque de 40 questions QI préremplies via seed.

Types de sous-sections :

- `logical_matrix` — Matrices logiques
- `numerical_series` — Séries numériques
- `verbal_analogies` — Analogies verbales
- `spatial_rotation` — Rotation spatiale

4.8 Autres tables

Table	Description	Clés étrangères
qi_subsections	Sous-sections de chaque test QI (4 par test)	qi_tests
qi_answers	Réponses données aux questions QI	qi_tests, qi_questions
dream_job_analyses	Analyse du métier rêvé (NLP simulé)	users
subscriptions	Historique des abonnements premium	users
career_roadmaps	Feuilles de route carrière générées	users
skill_gaps	Écarts de compétences niveau actuel vs requis	users, career_roadmaps
reports	Rapports PDF générés	users
market_data	Données du marché du travail (23 métiers seed)	indépendante

4.9 Diagramme des relations

```
users (1) ??hasOne??? (0..1) user_profiles
users (1) ??hasMany??? (0..n) academic_records
users (1) ??hasMany??? (0..n) personality_tests (1) ??hasMany??? personality_trait_results
users (1) ??hasMany??? (0..n) qi_tests (1) ??hasMany??? qi_subsections
??hasMany??? qi_answers
qi_answers ??belongsTo??? qi_questions
users (1) ??hasMany??? dream_job_analyses
users (1) ??hasMany??? subscriptions
users (1) ??hasMany??? reports
users (1) ??hasMany??? career_roadmaps (1) ??hasMany??? skill_gaps
market_data (table indépendante, utilisée par l'algorithme de matching)
```

5. Liste complète des endpoints API

Légende : Public Auth Premium

5.1 Authentification

Méthode	URL	Auth	Description	Paramètres
POST	/api/register	Public	Inscription	name, email, password, password_confirmation
POST	/api/login	Public	Connexion	email, password
POST	/api/logout	Auth	Déconnexion	-
GET	/api/me	Auth	Utilisateur courant	-

5.2 Profil utilisateur

Méthode	URL	Auth	Description
GET	/api/profile	Auth	Récupérer le profil
POST	/api/profile	Auth	Créer le profil
PUT	/api/profile	Auth	Mettre à jour le profil

Champs : bio, age, date_of_birth, gender, city, region, country, budget, budget_currency, target_job_title, target_job_description

5.3 Parcours académique

Méthode	URL	Auth	Description
GET	/api/academic-records	Auth	Lister les diplômes
POST	/api/academic-records	Auth	Ajouter un diplôme
GET	/api/academic-records/{id}	Auth	Détail d'un diplôme
PUT	/api/academic-records/{id}	Auth	Modifier un diplôme
DELETE	/api/academic-records/{id}	Auth	Supprimer un diplôme

Champs : diploma (req), field_of_study (req), institution, grade, start_date, end_date, disliked_subjects (array), sort_order

5.4 Tests de personnalité

Méthode	URL	Auth	Description
POST	/api/personality-tests/start	Auth	Démarrer un test (ou reprendre)
POST	/api/personality-tests/{id}/answers	Auth	Soumettre les réponses (min 10 traits)
GET	/api/personality-tests/{id}/results	Auth	Résultats d'un test
GET	/api/personality-tests/latest	Auth	Dernier test complété

Format de soumission :

```
{"traits": [{"trait_name": "Ouverture", "score": 75, "details": {}}, ...]}
```

Le overall_score est calculé automatiquement (moyenne des traits).

5.5 Tests QI

Méthode	URL	Auth	Description
POST	/api/qi-tests/start	Auth	Démarrer un test QI (4 sous-sections)
POST	/api/qi-tests/{id}/subsection/{subId}/answers	Auth	Soumettre les réponses
GET	/api/qi-tests/{id}/results	Auth	Résultats complets
GET	/api/qi-tests/latest	Auth	Dernier test complété

Format de soumission :

```
{"answers": [{"question_id": 1, "selected_answer": "A", "time_spent_seconds": 30}],
"time_spent_seconds": 120}
```

Calcul du QI : IQ = (pourcentage_bonnes_réponses / 100) × 60 + 70

5.6 Analyse du métier rêvé

Méthode	URL	Auth	Description
POST	/api/dream-job/analyze	Auth	Analyser un métier rêvé
GET	/api/dream-job/latest	Auth	Dernière analyse

Champs : job_title (req), job_description (req, min:10)

5.7 Orientation et carrière

Méthode	URL	Auth	Description
GET	/api/career/match	Auth	Matching carrière (top 15 métiers)
GET	/api/career/coherence	Auth	Vérifier la cohérence économique
POST	/api/career/roadmap	Premium	Générer une feuille de route
GET	/api/career/roadmaps	Auth	Lister les feuilles de route
GET	/api/career/skill-gaps	Auth	Lister les écarts de compétences

Le matching utilise : profil, personnalité, QI, marché, budget, localisation.

5.8 Abonnements

Méthode	URL	Auth	Description
POST	/api/subscriptions/activate	Auth	Activer un abonnement premium
GET	/api/subscriptions/status	Auth	Statut de l'abonnement

Paramètres activation : payment_method (req), transaction_id (req), duration_months (1, 3, 6 ou 12)

5.9 Rapports PDF

Méthode	URL	Auth	Description
GET	/api/reports	Auth	Lister les rapports
POST	/api/reports/generate/{type}	Auth	Générer un rapport
GET	/api/reports/{id}/download	Premium	Télécharger le PDF

Types de rapports :

- `initial` — Rapport de base (gratuit) : profil + matching
- `qi` — Rapport cognitif (premium) : résultats QI + personnalité
- `complete` — Rapport complet (premium) : tout + analyse SWOT

5.10 Assistant IA

Méthode	URL	Auth	Description
POST	/api/assistant/counsel	Premium	Conseil personnalisé
GET	/api/assistant/courses	Premium	Recommandations de formations

Paramètres counsel : message (req, min:2, max:2000 caractères)

6. Schémas des requêtes et réponses

6.1 Format standard

Toutes les réponses de l'API suivent ce format :

```
// Succès (200 / 201)
{
    "message": "Opération réussie.",
    "data": { ... }
}

// Erreur (4xx / 5xx)
{
    "message": "Message d'erreur.",
    "errors": { "field": ["Erreur de validation"] }
}
```

6.2 Exemple : GET /api/me

```
{
    "id": 1,
    "name": "Jean Dupont",
    "email": "jean@example.com",
    "subscription_status": "free",
    "subscription_expires_at": null,
    "profile": {
        "age": 22,
        "city": "Paris",
        "region": "Île-de-France",
        "target_job_title": "Développeur Full Stack"
    },
    "academic_records": [
        {
            "diploma": "Licence Informatique",
            "field_of_study": "Informatique",
            "institution": "Université Paris-Saclay",
            "grade": "Bien"
        }
    ]
}
```

6.3 Exemple : GET /api/career/match

```
{
    "matches": [
        {
            "job_title": "Data Scientist",
            "sector": "Tech & Numérique",
            "match_score": 87.5,
            "salary_avg": 55000,
            "tension_rate": 0.75,
            "factors": [
                {"name": "Localisation", "score": 15},
                {"name": "Budget", "score": 20}
```

```
            ],
            "alerts": []
        }
    ]
}
```

7. Codes d'erreur

Code	Statut HTTP	Description
400	Bad Request	Paramètres invalides
401	Unauthorized	Token manquant ou invalide
402	Premium Required	Abonnement premium requis (code: PREMIUM_REQUIRED)
403	Forbidden	L'utilisateur n'est pas propriétaire
404	Not Found	Ressource introuvable
422	Unprocessable Entity	Erreur de validation
429	Too Many Requests	Trop de requêtes
500	Internal Server Error	Erreur serveur

Erreur 402 — Premium Required

```
{
    "message": "Cette fonctionnalité nécessite un abonnement premium.",
    "code": "PREMIUM_REQUIRED"
}
```

8. Abonnements (Premium)

La plateforme propose deux niveaux d'accès : gratuit (free) et premium.

8.1 Fonctionnalités gratuites

- Inscription et gestion de profil
- Tests de personnalité complets
- Tests de QI complets
- Analyse du métier rêvé
- Matching carrière et cohérence économique
- Rapport initial (PDF)

8.2 Fonctionnalités premium

- Feuille de route carrière détaillée
- Rapport QI (PDF)
- Rapport complet avec analyse SWOT (PDF)
- Téléchargement des rapports (PDF)
- Assistant IA conseil personnalisé
- Recommandations de formations

8.3 Middleware subscription.active

Les routes protégées par le middleware `subscription.active` retournent une erreur **402** (PREMIUM_REQUIRED) si l'utilisateur n'a pas un abonnement premium actif. La vérification se fait sur les champs `subscription_status` et `subscription_expires_at` de la table `users`.

Annexe : Bonnes pratiques pour l'intégration

- Stockez le token dans un endroit sécurisé (httpOnly cookie de préférence, ou localStorage avec précautions).
- Incluez toujours le header `Accept: application/json` pour que Laravel retourne des erreurs JSON.
- Les dates sont au format ISO 8601 (ex: 2026-07-01T14:30:00Z).
- Les champs JSON (summary, details, disliked_subjects) sont automatiquement parsés en tableaux/objets par Laravel.
- Pour les tests QI, 40 questions sont préremplies dans la base via un seed.
- Pour toute question, contactez l'équipe backend.

— Fin du document —